

Учебник по проекту "Генерация и анализ ландшафта на основе гауссовых функций с клиент-серверной архитектурой"

Оглавление

1. Введение и обзор проекта
2. Архитектура программного обеспечения
3. Математические основы
4. Детальное описание классов и модулей
5. Алгоритмы и методы анализа
6. Форматы входных и выходных файлов
7. Процесс компиляции и выполнения
8. Практические примеры и анализ результатов
9. Заключение

Глава 1. Введение и обзор проекта

1.1. Назначение проекта

Данный проект представляет собой инструмент для генерации искусственного ландшафта, его визуализации и комплексного анализа. Проект разработан мною в рамках учебной работы по ЭВМ (Щёлокова Софья группа 212) и демонстрирует применение алгоритмов компьютерной графике, вычислительной геометрии, машинного обучения и системного программирования.

Основное назначение системы — создание реалистичного рельефа местности на основе суперпозиции гауссовых (нормальных) распределений, что позволяет моделировать природные формы: холмы, впадины, горные хребты и равнины. Сгенерированный ландшафт подвергается анализу, включающему вычисление градиентов, определение крутых участков, кластеризацию методом

К-средних, поиск связных компонент и построение геометрических структур — триангуляции Делоне и диаграммы Вороного.

Есть реализация клиент-серверной архитектуры с использованием именованных каналов (Named Pipes) в операционной системе Windows. Это обеспечивает четкое разделение между пользовательским интерфейсом (клиентом) и вычислительным ядром (сервером), что открывает возможности для распределенных вычислений, удаленного управления процессом генерации и потенциального масштабирования системы.

1.2. Основные функциональные возможности

Проект предоставляет следующий набор функциональных возможностей:

1. Генерация ландшафта:

- Добавление произвольного количества гауссовых колоколов с настраиваемыми параметрами (положение, дисперсия по осям, корреляция)
- Поддержка положительных (возвышенности) и отрицательных (впадины) компонент
- Нормализация высот в диапазон $[0, 255]$ для удобства визуализации
- Добавление контролируемого шума для повышения реалистичности
- Сглаживание рельефа гауссовым фильтром

2. Визуализация:

- Сохранение карты высот в формате BMP (8-битное полутоновое изображение)
- Построение трехмерной визуализации с использованием gnuplot
- Создание двумерной тепловой карты (heatmap)
- Построение градиента
- Визуализация геометрических структур (триангуляция Делоне, диаграмма Вороного)

3. Анализ рельефа:

- Вычисление градиентов во всех точках ландшафта
- Определение углов наклона поверхности
- Создание карты крутизны склонов с настраиваемым порогом
- Статистический анализ распределения высот

4. Кластеризация и поиск объектов:

- Кластеризация высоких участков методом К-средних
- Поиск связных компонент на основе пороговой обработки
- Фильтрация "шумовых" компонент малого размера

5. Геометрический анализ:

- Построение триангуляции Делоне по центрам найденных компонент
- Генерация диаграммы Вороного
- Визуализация геометрических структур

6. Системные возможности:

- Клиент-серверное взаимодействие через именованные каналы
- Автоматический запуск сервера при отсутствии соединения
- Подробное логирование системных и пользовательских событий
- Чтение команд из текстового файла
- Конфигурирование через внешний файл настроек

1.3. Технологический стек

Проект реализован на языке программирования C++ и использует следующие технологии и библиотеки:

- Язык программирования: C++ (стандарт C++11 и выше)
- Операционная система: Microsoft Windows
- Межпроцессное взаимодействие: Windows Named Pipes API
- Визуализация: Gnuplot (внешняя утилита)
- Математические операции: Стандартная библиотека C++ (cmath, random, algorithm)
- Работа с файлами: Стандартные потоки ввода-вывода C++
- Многопоточность: Стандартная библиотека thread, atomic
- Логирование: Пользовательская система с временными метками

1.4. Структура документации

Учебник представляет собой руководство по проекту, охватывающее все аспекты его функционирования — от теоретических основ до практических примеров использования. Документ структурирован таким образом, чтобы обеспечить последовательное погружение в материал: от общего обзора к

детальному рассмотрению отдельных компонентов, от теоретических концепций к их программной реализации.

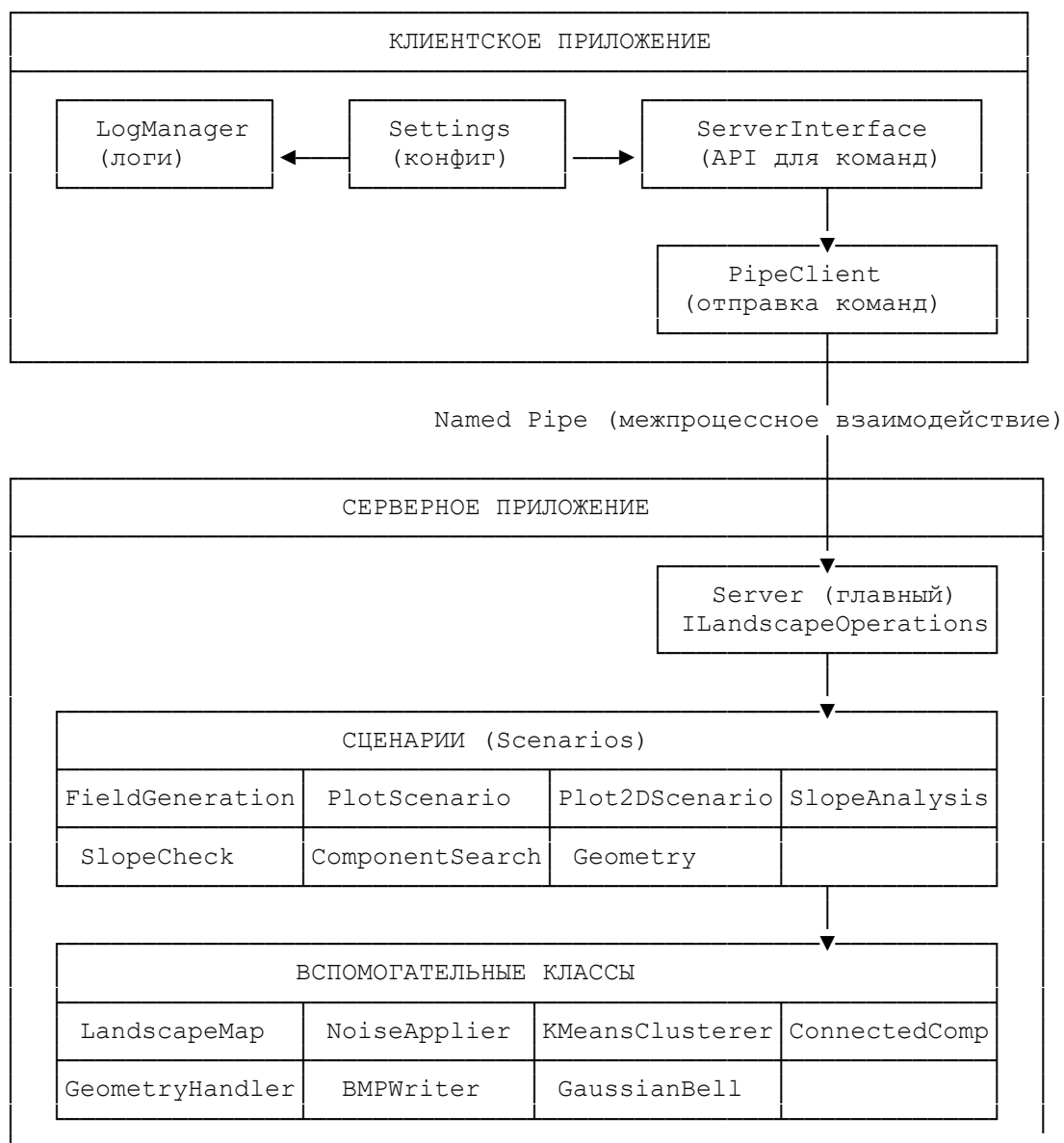
Каждая глава содержит подробные объяснения, фрагменты кода с комментариями, математические формулы и иллюстративные примеры.

Глава 2. Архитектура программного обеспечения

2.1. Общая архитектурная схема

Программный комплекс построен на основе клиент-серверной архитектуры с использованием именованных каналов Windows (Named Pipes) в качестве транспортного механизма. Основные удобства:

1. Четкое разделение ответственности: Клиент отвечает только за взаимодействие с пользователем и отправку команд, сервер — за всю вычислительную работу.
2. Возможность повторного использования: Один экземпляр сервера может обслуживать множество клиентских сессий.
3. Устойчивость к сбоям: Клиент и сервер работают в разных процессах, что предотвращает полное падение системы при ошибке в одном из компонентов.
4. Потенциал для распределенных вычислений: Архитектура допускает размещение клиента и сервера на разных машинах (при переходе на сетевые каналы).
5. Удобство отладки и тестирования: Компоненты могут тестироваться независимо друг от друга.



2.2. Взаимодействие клиента и сервера

Взаимодействие между клиентом и сервером осуществляется через именованный канал с именем `\\.\pipe\TerrainPipe`. Протокол обмена: клиент отправляет текстовую команду, сервер выполняет соответствующее действие и возвращает подтверждение ("OK").

2.2.1. Формат команд

Все команды представляют собой текстовые строки в следующем формате:

КОМАНДА [ПАРАМЕТР1] [ПАРАМЕТР2] ...

Поддерживаемые команды:

Команда - Параметры - Описание

GAUSS - sign cx cy sx sy rho - Добавление гауссова колокола
GENERATE - нет - Генерация ландшафта
PLOT - нет - Создание 3D-визуализации
PLOT2D - нет - Создание 2D-тепловой карты
BMP_WRITE - [filename] - Сохранение в BMP
ANALIZ - нет - Анализ градиентов
SLOPE_CHECK - нет - Проверка крутизны
COMPONENT_SEARCH - нет - Поиск компонент
GEOMETRY - нет - Геометрический анализ

2.2.2. Алгоритм работы PipeClient

Класс PipeClient реализует следующую логику подключения:

1. Попытка подключения к существующему серверу:
 - Вызов CreateFileA для открытия именованного канала
 - При ошибке ERROR_PIPE_BUSY — ожидание освобождения канала (до 1 секунды)
 - При успешном подключении — отправка команды и чтение ответа
2. Автоматический запуск сервера
 - Если после нескольких попыток подключиться не удалось, клиент запускает серверный процесс
 - Запуск осуществляется через CreateProcessW с параметром "server"
 - После запуска клиент ожидает 1 секунду для инициализации сервера
3. Обработка ошибок:
 - При превышении максимального числа попыток (30) возвращается сообщение об ошибке
 - Все ошибки логируются

```
// Упрощенная схема работы PipeClient::send()
string PipeClient::send(const string& msg) {
    for (int attempt = 0; attempt < 30; ++attempt) {
```

```

HANDLE hPipe = CreateFileA(PIPE_NAME, ...);

if (hPipe != INVALID_HANDLE_VALUE) {
    // Отправка команды
    WriteFile(hPipe, msg.c_str(), msg.length(), ...);

    // Чтение ответа
    char buffer[256];
    ReadFile(hPipe, buffer, sizeof(buffer), ...);

    CloseHandle(hPipe);
    return string(buffer);
}

if (!g_serverStarted) {
    // Запуск сервера при первой неудаче
    g_serverStarted = true;
    CreateProcessW(..., L"server");
    Sleep(1000);
}
}
return "ERROR: No server";
}

```

2.2.3. Серверный цикл обработки

Сервер работает в бесконечном цикле, обрабатывая подключения клиентов:

1. Создание именованного канала:

```

HANDLE hPipe = CreateNamedPipeA(
    PIPE_NAME.c_str(),
    PIPE_ACCESS_DUPLEX,    // Двусторонний обмен
    PIPE_TYPE_BYTE | PIPE_READMODE_BYTE | PIPE_WAIT,
    1,                    // Максимум 1 экземпляр
    4096, 4096,           // Размеры буферов
    0, NULL
);

```
2. Ожидание подключения клиента:

```

ConnectNamedPipe(hPipe, NULL);

```

3. Цикл обработки команд от клиента:
 - Чтение команды через ReadFile
 - Парсинг и выполнение команды
 - Отправка подтверждения через WriteFile
 - При разрыве соединения — выход из цикла
4. Завершение сессии:
 - DisconnectNamedPipe(hPipe);
 - CloseHandle(hPipe);

2.3. Паттерны проектирования

Использованы следующие паттерны проектирования:

2.3.1. Интерфейс (Interface)

ILandscapeOperations определяет контракт для всех операций с ландшафтом:

```
class ILandscapeOperations {  
public:  
    virtual ~ILandscapeOperations() = default;  
    virtual void addGauss(int sign, double cx, double cy,  
                        double sx, double sy, double rho) = 0;  
    virtual void generate() = 0;  
    virtual void plot() = 0;  
};
```

Класс Server реализует этот интерфейс, что позволяет при необходимости легко заменить реализацию или создать заглушку для тестирования.

2.3.2. Фасад

ServerInterface предоставляет упрощенный интерфейс для клиентской части, скрывая сложность взаимодействия с именованными каналами:

```
class ServerInterface {  
public:  
    void gauss(int s, double x, double y, double sx, double sy, double r) {
```



```

        ostream oss;
        oss << "GAUSS " << s << " " << x << " " << y << " "
            << sx << " " << sy << " " << r;
        PipeClient::send(oss.str());
    }
    // ...
};

```

2.3.3. Стратегия

Сценарии (FieldGenerationScenario, PlotScenario и др.) реализуют паттерн "Стратегия", инкапсулируя различные алгоритмы обработки ландшафта. Каждый сценарий содержит статический метод execute, принимающий необходимые параметры.

2.3.4. Одиночка

LogManager реализован как глобальный объект, доступный из любой части программы:

```
LogManager log_mgr; // Глобальный экземпляр
```

Это обеспечивает единую точку логирования для всего приложения.

2.4. Жизненный цикл приложения

Рассмотрим жизненный цикл приложения на примере типичного запуска:

1. Запуск клиента (без параметров):
 - Инициализация LogManager с ролью "client"
 - Загрузка конфигурации из config.txt
 - Чтение команд из commands.txt
2. Первая команда GAUSS:
 - ServerInterface::gauss() формирует текстовую команду
 - PipeClient::send() пытается подключиться к серверу
 - Сервер не найден → автоматический запуск серверного процесса
 - Сервер инициализируется, создает канал и ожидает подключения
 - Клиент подключается, отправляет команду, получает "OK"

3. Последующие команды:

- Клиент последовательно отправляет все команды из файла
- Сервер обрабатывает каждую команду в отдельной сессии (после каждой команды соединение закрывается)

4. Завершение работы:

- Клиент выводит список созданных файлов
- Сервер продолжает работать, ожидая новых подключений
- Пользователь может закрыть сервер вручную (нажав Enter в его консоли)

2.5. Многопоточность и синхронизация

В текущей реализации сервер обрабатывает подключения последовательно, однако архитектура допускает расширение до многопоточной обработки. Для этого потребуется:

1. Создавать новый поток для каждого подключения
2. Использовать мьютексы для синхронизации доступа к разделяемым данным (LandscapeMap)

Пример потенциальной многопоточной реализации:

```
void Server::runPipeServer() {
    while (running) {
        HANDLE hPipe = CreateNamedPipeA(...);
        ConnectNamedPipe(hPipe, NULL);

        // Запуск потока для обработки клиента
        thread client_thread([this, hPipe]() {
            handleClient(hPipe);
            DisconnectNamedPipe(hPipe);
            CloseHandle(hPipe);
        });
        client_thread.detach();
    }
}
```

Глава 3. Математические основы

3.1. Двумерное нормальное распределение (гауссова функция)

Гауссова функция является фундаментальным блоком для генерации ландшафта в данном проекте. Она описывает плавное возвышение или впадину, форма которой напоминает колокол.

3.1.1. Общая формула

Двумерная функция плотности нормального распределения с учетом корреляции между координатами:

$$G(x, y) = \frac{1}{2\pi\sigma_x\sigma_y\sqrt{1-\rho^2}} \exp\left(-\frac{1}{2(1-\rho^2)} \left[\frac{(x-\mu_x)^2}{\sigma_x^2} + \frac{(y-\mu_y)^2}{\sigma_y^2} - \frac{2\rho(x-\mu_x)(y-\mu_y)}{\sigma_x\sigma_y} \right]\right)$$

В проекте используется упрощенная форма без нормировочного коэффициента:

$$G_{proj}(x, y) = \text{sign} \cdot \exp\left(-\frac{1}{2(1-\rho^2)} \left[\left(\frac{x-\mu_x}{\sigma_x}\right)^2 + \left(\frac{y-\mu_y}{\sigma_y}\right)^2 - 2\rho\frac{x-\mu_x}{\sigma_x}\frac{y-\mu_y}{\sigma_y} \right]\right)$$

где:

- μ_x, μ_y — координаты центра колокола
- σ_x, σ_y — стандартные отклонения по осям
- ρ — коэффициент корреляции между координатами
- $\{-1, 1\}$ — знак колокола (1 — возвышение, -1 — впадина)

3.1.2. Влияние параметров на форму колокола

Стандартные отклонения (σ_x, σ_y):

- Определяют "размах" колокола по соответствующим осям
- Большие значения создают пологие, широкие холмы
- Малые значения создают крутые, узкие пики
- Разные значения по осям позволяют создавать вытянутые формы (хребты)

Коэффициент корреляции ($\rho \in (-1, 1)$):

- Определяет поворот и деформацию колокола
- $\rho = 0$ — симметричный колокол без поворота

- $\rho > 0$ — положительная корреляция, колокол вытянут вдоль прямой $y = x$
- $\rho < 0$ — отрицательная корреляция, колокол вытянут вдоль прямой $y = -x$
- Значения, близкие к ± 1 , создают сильно вытянутые формы

Знак:

- +1 — стандартный колокол (холм, возвышенность)
- -1 — перевернутый колокол (впадина, кратер)

3.1.3. Реализация в коде

```
struct GaussianBell {
    int sign;
    double cx, cy, sx, sy, rho;

    double value(double x, double y) const {
        double dx = x - cx, dy = y - cy;
        double nx = dx / sx, ny = dy / sy;
        double denom = 1 - rho * rho;
        if (fabs(denom) < 1e-10) return 0.0;
        double q = nx*nx + ny*ny - 2*rho*nx*ny;
        return sign * exp(-0.5 * q / denom);
    }
};
```

Обращу внимание на проверку $\text{fabs}(\text{denom}) < 1e-10$ — она предотвращает деление на ноль при ρ примерно равное ± 1

3.2. Суперпозиция гауссовых функций

Итоговая карта высот формируется как сумма значений всех гауссовых колоколов:

$H(x,y) = \sum_{i=1}^N G_i(x,y)$, где G_i — i -й гауссов колокол со своими параметрами.

После суммирования выполняется нормализация в диапазон [0, 255]:

$$H_{\text{norm}}(x,y) = 255 * \frac{H(x,y) - H_{\min}}{H_{\max} - H_{\min}}, \text{ где } H_{\min} = \min_{x,y} H(x,y) \text{ и } H_{\max} = \max_{x,y} H(x,y)$$

3.3. Вычисление градиента

Градиент функции высот — вектор, указывающий направление наибольшего роста функции:

$$\nabla H(x, y) = \left(\frac{\partial H}{\partial x}, \frac{\partial H}{\partial y} \right)$$

В дискретной реализации используется центральная разностная схема второго порядка точности:

$$\frac{\partial H}{\partial x}(x, y) \approx \frac{H(x+1, y) - H(x-1, y)}{2}$$

$$\frac{\partial H}{\partial y}(x, y) \approx \frac{H(x, y+1) - H(x, y-1)}{2}$$

Крутизна склона (модуль градиента):

$$S(x, y) = \sqrt{\left(\frac{\partial H}{\partial x}\right)^2 + \left(\frac{\partial H}{\partial y}\right)^2}$$

Угол наклона:

$$\theta(x, y) = \arctan\left(\frac{\partial H}{\partial y}, \frac{\partial H}{\partial x}\right)$$

3.3.1. Реализация в коде

```
pair<double, double> LandscapeMap::gradient(int x, int y) const {
    if (x <= 0 || x >= w - 1 || y <= 0 || y >= h - 1)
        return {0.0, 0.0}; // Граничные точки
    double dx = (data[y][x + 1] - data[y][x - 1]) / 2.0;
    double dy = (data[y + 1][x] - data[y - 1][x]) / 2.0;
    return {dx, dy};
}
```

3.4. Сглаживание гауссовым фильтром

Для устранения резких перепадов и игл на картинке я применила сглаживание с использованием гауссова ядра 3×3 :

$$K = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \quad (\text{разделимое ядро})$$

Свертка выполняется по формуле:

$$H_{smooth}(x, y) = \frac{\sum_{dy=-1}^1 \sum_{dx=-1}^1 K[dy+1][dx+1] \cdot H(x+dx, y+dy)}{\sum_{dy=-1}^1 \sum_{dx=-1}^1 K[dy+1][dx+1]}$$

У меня сглаживание применяется дважды для достижения более сильного эффекта.

3.5. Алгоритм K-средних (K-means)

K-средних — итеративный алгоритм кластеризации, разделяющий множество точек на K кластеров. В проекте используется для группировки высоких участков ландшафта.

3.5.1. Математическая формулировка

Входные данные:

- Множество точек $P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Количество кластеров K

Целевая функция:

Минимизация суммы квадратов расстояний от точек до центров их кластеров:

$$J = \sum_{i=1}^K \sum_{p \in C_i} \|p - \mu_i\|^2$$

где:

- C_i — множество точек i -го кластера
- μ_i — центр i -го кластера

Алгоритм:

1. Инициализация: Случайный выбор K точек в качестве начальных центроидов

2. Шаг присваивания: Каждая точка относится к ближайшему центроиду:

$$C_i^{(t)} = \{p : \|p - \mu_i^{(t)}\| \leq \|p - \mu_j^{(t)}\| \text{ для всех } j\}$$

3. Шаг обновления: Пересчет центроидов как средних арифметических точек в кластере:

$$\mu_i^{(t+1)} = \frac{1}{|C_i^{(t)}|} \sum_{p \in C_i^{(t)}} p$$

4. Проверка сходимости: Если назначения кластеров не изменились — алгоритм завершен, иначе повтор с шага 2

3.5.2. Реализация в проекте

```
vector<int> KMeansClusterer::cluster(const LandscapeMap& map, int k,
int min_size) {
    // Отбор точек с высотой >= 128
    vector<pair<double, double>> points;
    vector<int> point_idx;
    for (int y = 0; y < map.h; y++)
        for (int x = 0; x < map.w; x++)
            if (map.data[y][x] >= 128) {
                points.emplace_back(x, y);
                point_idx.push_back(y * map.w + x);
            }

    int np = points.size();
    k = min(k, np);

    // Случайная инициализация центроидов
    vector<pair<double, double>> centroids(k);
    for (int i = 0; i < k; i++) {
        int ri = rand() % np;
        centroids[i] = points[ri];
    }
}
```

```

// Итеративный процесс
bool changed = true;
int maxIter = 50;
while (changed && maxIter--) {
    changed = false;
    vector<vector<int>> r(k, vector<int>(np, 0));

    // Шаг присваивания
    for (int i = 0; i < np; i++) {
        int best = 0;
        double md = 1e9;
        for (int c = 0; c < k; c++) {
            double d = pow(points[i].first - centroids[c].first, 2) +
                pow(points[i].second - centroids[c].second, 2);
            if (d < md) {
                md = d;
                best = c;
            }
        }
        r[best][i] = 1;
        if (labels[point_idx[i]] != best) {
            changed = true;
            labels[point_idx[i]] = best;
        }
    }

    // Шаг обновления
    for (int c = 0; c < k; c++) {
        double sx = 0.0, sy = 0.0, sum_r = 0.0;
        for (int i = 0; i < np; i++) {
            if (r[c][i]) {
                sx += points[i].first;
                sy += points[i].second;
                sum_r += 1.0;
            }
        }
        if (sum_r >= min_size && sum_r > 0) {
            centroids[c] = {sx / sum_r, sy / sum_r};
        }
    }
}
return labels;
}

```


3.6. Поиск связанных компонент

Алгоритм поиска связных компонент на основе обхода в ширину (BFS) используется для выделения областей с высотой выше порогового значения.

3.6.1. Определение связности

Используется 8-связность — пиксели считаются соседними, если они соприкасаются сторонами или углами:

[-1,-1] [0,-1] [1,-1]
 [-1, 0] [x, y] [1, 0]
 [-1, 1] [0, 1] [1, 1]

3.6.2. Алгоритм BFS

[illegible]

```

        if (nx >= 0 && nx < map.w && ny >= 0 && ny < map.h
&&
            map.data[ny][nx] > 200 && !visited[ny][nx]) {
                visited[ny][nx] = true;
                q.push({nx, ny});
            }
        }
    }

    // Фильтрация по минимальному размеру
    if (comp.size() >= static_cast<size_t>(min_size)) {
        components.push_back(comp);
    } else {
        // Игнорирование "пыли"
        log_mgr.log_system("Dust ignored: component size = " +
            to_string(comp.size()));
    }
}
}
}
return components;
}

```

3.7. Триангуляция Делоне

Триангуляция Делоне — это разбиение множества точек на треугольники, обладающее свойством: внутри описанной окружности любого треугольника нет других точек множества.

3.7.1. Свойства триангуляции Делоне

1. Максимизация минимального угла: Среди всех возможных триангуляций, триангуляция Делоне максимизирует минимальный угол треугольников (избегает "тонких" треугольников)
2. Двойственность с диаграммой Вороного: Центры описанных окружностей треугольников Делоне являются вершинами диаграммы Вороного
3. Уникальность: Для общего положения точек (никакие четыре не лежат на одной окружности) триангуляция Делоне единственна

3.7.2. Вычисление описанной окружности

Для треугольника с вершинами (x_1, y_1) , (x_2, y_2) , (x_3, y_3) центр описанной окружности (x_c, y_c) вычисляется по формулам:

$$D = 2(x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2))$$

$$x_c = \frac{(x_1^2 + y_1^2)(y_2 - y_3) + (x_2^2 + y_2^2)(y_3 - y_1) + (x_3^2 + y_3^2)(y_1 - y_2)}{D}$$

$$y_c = \frac{(x_1^2 + y_1^2)(x_3 - x_2) + (x_2^2 + y_2^2)(x_1 - x_3) + (x_3^2 + y_3^2)(x_2 - x_1)}{D}$$

Радиус описанной окружности:

$$R = \sqrt{(x_1 - x_c)^2 + (y_1 - y_c)^2}$$

3.7.3. Алгоритм "в лоб"

Реализован простой алгоритм перебора всех возможных треугольников:

```
void GeometryHandler::buildDelaunay() {
    int n = points.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            for (int k = j + 1; k < n; k++) {
                Triangle tri(i, j, k, points);
                bool isDelaunay = true;

                // Проверка условия Делоне
                for (int m = 0; m < n; m++) {
                    if (m == i || m == j || m == k) continue;
                    if (tri.containsPoint(points[m])) {
                        isDelaunay = false;
                        break;
                    }
                }

                if (isDelaunay) {
                    triangles.push_back(tri);
                }
            }
        }
    }
}
```

```

    }
  }
}
}

```

Сложность алгоритма: $O(n^4)$, где n — количество точек. Для $n = 8$ (как в примере) требуется проверить биномиальный коэффициент из 8 по 3 = 56 треугольников, каждый из которых проверяется на 5 оставшихся точек — всего около 280 проверок.

3.7.4. Проверка принадлежности точки описанной окружности

```

bool Triangle::containsPoint(const Point2D& p) const {
    return circumcenter.distance(p) < circumradius - 1e-9;
}

```

3.8. Диаграмма Вороного

Диаграмма Вороного разбивает плоскость на области, каждая из которых состоит из точек, ближайших к одному из заданных центров.

3.8.1. Связь с триангуляцией Делоне

Для построения диаграммы Вороного используется двойственность с триангуляцией Делоне:

- Ребра диаграммы Вороного соединяют центры описанных окружностей смежных треугольников Делоне
- Два треугольника считаются смежными, если они имеют общее ребро (две общие вершины)

3.8.2. Реализация

```

// Построение ребер Вороного
for (size_t i = 0; i < triangles.size(); i++) {
    for (size_t j = i + 1; j < triangles.size(); j++) {
        int shared = 0;

```

```

// Подсчет общих вершин
if (triangles[i].p1 == triangles[j].p1 ||
    triangles[i].p1 == triangles[j].p2 ||
    triangles[i].p1 == triangles[j].p3) shared++;
if (triangles[i].p2 == triangles[j].p1 ||
    triangles[i].p2 == triangles[j].p2 ||
    triangles[i].p2 == triangles[j].p3) shared++;
if (triangles[i].p3 == triangles[j].p1 ||
    triangles[i].p3 == triangles[j].p2 ||
    triangles[i].p3 == triangles[j].p3) shared++;

if (shared == 2) { // Смежные треугольники
    // Соединяем их центры описанных окружностей
    vFile << triangles[i].circumcenter.x << " "
        << triangles[i].circumcenter.y << "\n";
    vFile << triangles[j].circumcenter.x << " "
        << triangles[j].circumcenter.y << "\n\n";
}
}
}

```

3.9. Генерация шума

Для повышения реалистичности ландшафта добавляется случайный шум:

```

void NoiseApplier::apply(LandscapeMap& map, double percent) {
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<double> dis(0.0, 1.0);

    for (int y = 0; y < map.h; y++) {
        for (int x = 0; x < map.w; x++) {
            if (dis(gen) < percent / 100.0) {
                // Добавление шума в диапазоне [-20, 20]
                map.data[y][x] += (dis(gen) - 0.5) * 40.0;
                map.data[y][x] = max(0.0, min(255.0, map.data[y][x]));
            }
        }
    }
}

```

Глава 4. Детальное описание классов и модулей

4.1. Иерархия классов

Проект содержит следующие основные классы:

ILandscapeOperations (интерфейс)

└─ Server (реализация)

LogManager (логирование)

PipeClient (клиент канала)

ServerInterface (фасад для клиента)

Settings (конфигурация)

LandscapeMap (карта высот)

GaussianBell (гауссов колокол)

BMPWriter (запись BMP)

NoiseApplier (добавление шума)

KMeansClusterer (кластеризация)

ConnectedComponentFinder (связные компоненты)

GeometryHandler (геометрия)

└─ Point2D

└─ Triangle

Сценарии:

FieldGenerationScenario

PlotScenario

Plot2DScenario

SlopeAnalysisScenario

SlopeCheckScenario

ComponentSearchScenario

GeometryScenario

4.2. LogManager — система логирования

LogManager обеспечивает централизованное логирование событий в два отдельных файла:

- Системный лог (server_log.log / client_log.log) — техническая информация
- Пользовательский лог (server_user.log / client_user.log) — информация для пользователя

4.2.1. Атрибуты и методы

```
class LogManager {  
    ofstream system_log, user_log;  
    bool enabled = true;  
  
public:  
    void setup(const string& role);  
    void log_system(const string& msg);  
    void log_user(const string& msg);  
};
```

4.2.2. Формат логов

Каждое сообщение предваряется временной меткой в формате ISO 8601:

```
[2026-04-18 18:38:23] SYSTEM: Added Gaussian bell at (400, 600)  
[2026-04-18 18:38:23] USER: Executing: GAUSS 1 400 600 80 80 0
```

4.2.3. Особенности реализации

- Логи одновременно выводятся в файл и в консоль
- При невозможности открыть файлы логирования, enabled устанавливается в false
- Деструктор автоматически закрывает файлы

4.3. Settings — управление конфигурацией

Класс Settings отвечает за загрузку и хранение параметров конфигурации из файла config.txt.

4.3.1. Параметры конфигурации

Параметр	Тип	По умолчанию	Описание
WIDTH	int	900	Ширина карты в пикселях
HEIGHT	int	900	Высота карты в пикселях
KMEANS_K	int	2	Количество кластеров для K-means
SLOPE_THRESHOLD	double	2.0	Порог крутизны склона
NOISE_LEVEL	double	5.0	Уровень шума в процентах
MIN_CLUSTER_SIZE	int	2	Минимальный размер кластера
MAX_K	int	5	Максимальное количество кластеров

4.3.2. Парсинг конфигурационного файла

```
bool Settings::load(const string& path) {  
    ifstream f(path);  
    if (!f.is_open()) return false;  
  
    string line;  
    while (getline(f, line)) {  
        if (line.empty() || line[0] == '#') continue;  
  
        size_t eq = line.find('=');  
        if (eq == string::npos) continue;  
  
        string key = line.substr(0, eq);  
        string val = line.substr(eq + 1);  
    }  
}
```



```

// Удаление пробелов
key.erase(0, key.find_first_not_of(" \t"));
key.erase(key.find_last_not_of(" \t\r\n") + 1);
val.erase(0, val.find_first_not_of(" \t"));
val.erase(val.find_last_not_of(" \t\r\n") + 1);

// Присваивание значений
if (key == "WIDTH") width = stoi(val);
else if (key == "HEIGHT") height = stoi(val);
// ... остальные параметры
}
return true;
}

```

4.4. *LandscapeMap* — карта высот

Центральный класс для хранения и обработки данных о высотах.

4.4.1. Структура данных

Карта высот хранится в виде двумерного вектора:

```
vector<vector<double>> data; // data[y][x]
```

Индексация: `data[y][x]`, где `y` — строка (`0..h-1`), `x` — столбец (`0..w-1`).

4.4.2. Основные методы

Метод - Описание:

`add(const GaussianBell& b)` - Добавление гауссова колокола к карте

`normalize()` - Нормализация высот в `[0, 255]`

`gradient(int x, int y)` - Вычисление градиента в точке

`saveBMP(const string& fn)` - Сохранение в BMP

4.4.3. Реализация нормализации

```

void LandscapeMap::normalize() {
    double mn = 1e9, mx = -1e9;

    // Поиск минимума и максимума
    for (auto& row : data)
        for (double v : row) {
            mn = min(mn, v);
            mx = max(mx, v);
        }

    double r = mx - mn;
    if (r < 1e-8) return; // Защита от деления на ноль

    // Линейное отображение в [0, 255]
    for (auto& row : data)
        for (double& v : row)
            v = (v - mn) / r * 255.0;
}

```

4.5. *GaussianBell* — гауссов колокол

Структура, представляющая один гауссов колокол.

```

struct GaussianBell {
    int sign;    // Знак: 1 (холм) или -1 (впадина)
    double cx, cy; // Координаты центра
    double sx, sy; // Стандартные отклонения
    double rho;   // Коэффициент корреляции

    GaussianBell(int s, double x, double y, double vx, double vy, double r)
        : sign(s), cx(x), cy(y), sx(vx), sy(vy), rho(r) {}

    double value(double x, double y) const;
};

```

4.6. *BMPWriter* — запись BMP-файлов

Класс для сохранения карты высот в формате BMP (8-битное полутоновое изображение).

4.6.1. Структура BMP-файла

BMP-файл состоит из следующих частей:

1. BITMAPFILEHEADER (14 байт):
 - Сигнатура "BM" (2 байта)
 - Размер файла (4 байта)
 - Зарезервировано (4 байта)
 - Смещение до данных (4 байта)
2. BITMAPINFOHEADER (40 байт):
 - Размер заголовка (4 байта)
 - Ширина и высота (по 4 байта)
 - Плоскости (2 байта)
 - Бит на пиксель (2 байта)
 - Сжатие (4 байта)
 - Размер изображения (4 байта)
 - Разрешение (8 байт)
 - Количество цветов (4 байта)
 - Важные цвета (4 байта)
3. Палитра (1024 байта для 8-бит):
 - 256 записей по 4 байта
4. Данные изображения:
 - Строки снизу вверх
 - Каждая строка выравнивается до кратности 4 байтам

4.6.2. Реализация

```
void BMPWriter::write(const LandscapeMap& map, const string&
filename) {
    ofstream f(filename, ios::binary);

    // Вычисление выравнивания
    int pad = (4 - (map.w % 4)) % 4;
    uint32_t imgSz = static_cast<uint32_t>((map.w + pad) * map.h);
    uint32_t fileSize = 54 + 256 * 4 + imgSz;

    // Запись заголовков
    uint8_t hdr[54] = {0x42, 0x4D}; // "BM"
    // ... заполнение заголовков

    // Запись палитры (градации серого)
```

```

for (int i = 0; i < 256; i++) {
    uint8_t c = static_cast<uint8_t>(i);
    f.write(reinterpret_cast<char*>(&c), 1); // B
    f.write(reinterpret_cast<char*>(&c), 1); // G
    f.write(reinterpret_cast<char*>(&c), 1); // R
    f.put(0); // Резерв
}

// Запись данных (снизу вверх)
for (int y = map.h - 1; y >= 0; y--) {
    for (int x = 0; x < map.w; x++) {
        uint8_t v = static_cast<uint8_t>(
            max(0.0, min(255.0, map.data[y][x]))
        );
        f.write(reinterpret_cast<char*>(&v), 1);
    }
    // Выравнивание
    for (int i = 0; i < pad; i++) f.put(0);
}
}

```

4.7. *NoiseApplier* — добавление шума

Статический класс для добавления случайного шума к карте высот.

```

class NoiseApplier {
public:
    static void apply(LandscapeMap& map, double percent);
};

```

Шум добавляется с вероятностью $\text{percent} / 100$ к каждому пикселю.
Амплитуда шума: $[-20, 20]$.

4.8. *KMeansClusterer* — кластеризация

Статический класс, реализующий алгоритм К-средних для кластеризации высоких участков ландшафта.

```

class KMeansClusterer {
public:

```

```
static vector<int> cluster(const LandscapeMap& map, int k, int
min_size);
};
```

Возвращает вектор меток кластеров для каждой точки карты. Точки с высотой < 128 получают метку -1 (не кластеризуются).

4.9. ConnectedComponentFinder — связанные компоненты

Статический класс для поиска связанных компонент на бинаризованной карте.

```
class ConnectedComponentFinder {
public:
    static vector<vector<pair<int, int>>> find(
        const LandscapeMap& map, int min_size
    );
};
```

Возвращает список компонент, каждая компонента — список координат пикселей.

4.10. GeometryHandler — геометрические построения

Класс для построения триангуляции Делоне и диаграммы Вороного.

```
class GeometryHandler {
    vector<Point2D> points;
    vector<Triangle> triangles;
    int width, height;

public:
    GeometryHandler(const vector<Point2D>& centers, int w, int h);
    void buildDelaunay();
    void visualize(const string& filename);
};
```

4.10.1. Point2D — точка на плоскости

```
struct Point2D {
    double x, y;
    double distance(const Point2D& other) const {
```

```

        return sqrt((x - other.x)*(x - other.x) +
                    (y - other.y)*(y - other.y));
    }
};

```

4.10.2. *Triangle* — треугольник

```

struct Triangle {
    int p1, p2, p3;          // Индексы вершин
    Point2D circumcenter;    // Центр описанной окружности
    double circumradius;     // Радиус описанной окружности

    Triangle(int i1, int i2, int i3, const vector<Point2D>& pts);
    bool containsPoint(const Point2D& p) const;
};

```

4.11. *Сценарии (Scenarios)*

Сценарии инкапсулируют отдельные операции обработки ландшафта. Все они реализованы как статические классы с методом `execute`.

4.11.1. *FieldGenerationScenario*

Генерация ландшафта: добавление гауссовых колоколов, нормализация, добавление шума, сглаживание.

```

class FieldGenerationScenario {
public:
    static void execute(LandscapeMap& map,
                       const vector<GaussianBell>& bells,
                       double noise_percent);
};

```

4.11.2. *PlotScenario*

Создание 3D-визуализации через `gnuplot`.

```

class PlotScenario {
public:
    static void execute(const LandscapeMap& map);
};

```

```
};
```

Формат данных для gnuplot:

```
x1 y1 z1
```

```
x2 y2 z2
```

```
...
```

```
x1 y2 z1
```

```
...
```

4.11.3. Plot2DScenario

Создание 2D-тепловой карты через gnuplot.

4.11.4. SlopeAnalysisScenario

Анализ градиентов: вычисление углов и модулей градиента, сохранение в файл.

4.11.5. SlopeCheckScenario

Создание бинарной карты крутизны склонов на основе порога.

4.11.6. ComponentSearchScenario

Кластеризация методом К-средних.

4.11.7. GeometryScenario

Построение триангуляции Делоне и диаграммы Вороного по центрам связанных компонент.

4.12. Server

Класс Server реализует интерфейс ILandscapeOperations и управляет всей вычислительной логикой.

```
class Server : public ILandscapeOperations {  
    vector<GaussianBell> bells;  
    LandscapeMap* map = nullptr;  
    Settings cfg;  
    atomic<bool> running{true};
```

```
public:
```

```

Server();
~Server();
void init();
void runPipeServer();

// Реализация ILandscapeOperations
void addGauss(...) override;
void generate() override;
void plot() override;
// ...
private:
    void executeCommand(const string& line);
};

```

4.13. *ServerInterface* — клиентский API

Фасад, предоставляющий удобный интерфейс для клиента.

```

class ServerInterface {
public:
    void gauss(int s, double x, double y, double sx, double sy, double r);
    void generate();
    void plot();
    void plot2D();
    void saveBMP(const string& name = "my_landscape.bmp");
    void analiz();
    void slopeCheck();
    void componentSearch();
    void geometry();
};

```

Глава 5. Алгоритмы и методы анализа

5.1. *Полный цикл обработки ландшафта*

Я покажу полный цикл обработки от команды GENERATE до получения финальных результатов.

5.1.1. *Этап 1: Генерация базового ландшафта*


```

void FieldGenerationScenario::execute(LandscapeMap& map,
                                     const vector<GaussianBell>& bells,
                                     double noise_percent) {
    // 1. Суперпозиция гауссовых колоколов
    for (const auto& b : bells) {
        map.add(b);
    }

    // 2. Нормализация в [0, 255]
    map.normalize();

    // 3. Добавление шума
    NoiseApplier::apply(map, noise_percent);

    // 4. Двойное сглаживание
    smooth(map);
    smooth(map);
}

```

5.1.2. Этап 2: Анализ градиентов

```

void SlopeAnalysisScenario::execute(const LandscapeMap& map) {
    ofstream slope("gradient_vectors.txt");
    int step = 20; // Шаг для разреженного вывода

    for (int y = step; y < map.h - step; y += step) {
        for (int x = step; x < map.w - step; x += step) {
            auto grad = map.gradient(x, y);
            double ang = atan2(grad.second, grad.first);
            double mag = sqrt(grad.first * grad.first +
                              grad.second * grad.second);
            slope << x << " " << y << " " << ang << " " << mag << "\n";
        }
    }
}

```

5.1.3. Этап 3: Карта крутизны

```

void SlopeCheckScenario::execute(const LandscapeMap& map, double
threshold) {
    LandscapeMap check(map.w, map.h);

    for (int y = 1; y < map.h - 1; y++) {

```

```

    for (int x = 1; x < map.w - 1; x++) {
        auto grad = map.gradient(x, y);
        double steep = sqrt(grad.first * grad.first +
                             grad.second * grad.second);
        // Бинаризация по порогу
        check.data[y][x] = (steep < threshold) ? 255 : 0;
    }
}

check.saveBMP("steepness_map.bmp");
}

```

5.1.4. Этап 4: Поиск компонент и геометрия

```

void GeometryScenario::execute(const LandscapeMap& map, int
min_size) {
    // 1. Поиск связанных компонент
    auto comps = ConnectedComponentFinder::find(map, min_size);

    // 2. Вычисление центров компонент
    vector<Point2D> centers;
    for (auto& comp : comps) {
        double sx = 0, sy = 0;
        for (auto& p : comp) {
            sx += p.first;
            sy += p.second;
        }
        centers.push_back({sx / comp.size(), sy / comp.size()});
    }

    // 3. Построение геометрии
    GeometryHandler gh(centers, map.w, map.h);
    gh.buildDelaunay();
    gh.visualize("my_delaunay_voronoi.png");
}

```

5.2. Сглаживание изображения

Функция `smooth()` реализует гауссово размытие с ядром 3×3 :

```

void smooth(LandscapeMap& map) {
    auto copy = map.data;
    double kernel[3][3] = {{1,2,1}, {2,4,2}, {1,2,1}};

    for (int y = 1; y < map.h - 1; y++) {
        for (int x = 1; x < map.w - 1; x++) {
            double sum = 0, wsum = 0;
            for (int dy = -1; dy <= 1; dy++) {
                for (int dx = -1; dx <= 1; dx++) {
                    double w = kernel[dy + 1][dx + 1];
                    sum += copy[y + dy][x + dx] * w;
                    wsum += w;
                }
            }
            map.data[y][x] = sum / wsum;
        }
    }
}

```

Свойства ядра:

- Сумма весов: $1+2+1+2+4+2+1+2+1 = 16$
- Ядро разделимое: может быть реализовано как две одномерные свертки для повышения производительности
- Эффективно подавляет высокочастотный шум

5.3. Визуализация через *gnuplot*

Gnuplot используется для создания визуализаций. Проект генерирует скрипты *gnuplot* и выполняет их через *system()*.

5.3.1. 3D-визуализация

Скрипт *my_plot_script.gnuplot*:

```

gnuplot
set terminal pngcairo size 1200,900
set output 'terrain_3d.png'
set title 'Terrain Field (3D)'
set pm3d
splot 'terrain_data.txt' with pm3d

```

5.3.2. 2D-тепловая карта

Скрипт my_plot_2d.gnuplot:

```
gnuplot
set terminal pngcairo size 900,900
set output 'terrain_2d.png'
plot 'terrain_data_2d.txt' with image
```

5.3.3. Визуализация геометрии

Скрипт plot_geometry.gnuplot:

```
gnuplot
set terminal pngcairo size 1000,1000
set output 'my_delaunay_voronoi.png'
set xrange [0:900]
set yrange [0:900]
set size square
plot 'delaunay.txt' with lines lc rgb 'blue' title 'Delaunay', \
     'voronoi.txt' with lines lc rgb 'red' lw 2 title 'Voronoi', \
     'points_centers.txt' with points pt 7 ps 2 lc rgb 'black' title 'Centers'
```

5.4. Анализ производительности

5.4.1. Сложность основных операций

Добавление гауссова колокола — сложность: «О от ширины на высоту». Это значит, что время работы прямо пропорционально количеству пикселей на изображении. Для каждого пикселя выполняется одна операция.

Нормализация — сложность: «О от ширины на высоту». Требуется два полных прохода по всем пикселям, но это всё равно линейная зависимость от их количества.

Сглаживание — сложность: «О от ширины на высоту». Для каждого пикселя усредняются значения в квадратике 3 на 3 вокруг него. Но так как размер окна фиксированный, общая сложность остаётся пропорциональной числу пикселей.

Вычисление градиентов — сложность: «О от ширины на высоту». Действие выполняется для каждой точки изображения.

К-средних (K-means) — сложность: «О от К умножить на N умножить на I». Где:

- K — количество кластеров (групп),
- N — количество точек,
- I — количество итераций (повторений алгоритма).

Поиск компонент связности — сложность: «О от ширины на высоту». Используется поиск в ширину (BFS), который обходит все пиксели.

Триангуляция Делоне — сложность: «О от n в четвёртой степени». Это очень медленно. n — количество центров компонент (точек, по которым строится триангуляция).

Итог для карты 900 на 900 пикселей (это 810 тысяч пикселей): Большинство операций выполняются за доли секунды, потому что они линейно зависят от количества пикселей. Самая затратная по времени операция — триангуляция Делоне, но на практике она работает с очень малым количеством точек (обычно меньше 20), поэтому даже её четвёртая степень не создаёт проблем.

5.4.2. Оптимизации

1. Локальность данных: Двумерный вектор обеспечивает кэш-локальность при построчном доступе
2. Раннее отсечение: В K-means точки с высотой < 128 не рассматриваются
3. Фильтрация компонент: Мелкие компоненты ("пыль") игнорируются

5.5. Обработка ошибок и устойчивость

Проект содержит несколько уровней защиты от ошибок:

1. Проверка границ массива:

```
if (x <= 0 || x >= w - 1 || y <= 0 || y >= h - 1)
    return {0.0, 0.0};
```

2. Защита от деления на ноль:

```
if (fabs(denom) < 1e-10) return 0.0;
if (r < 1e-8) return; // При нормализации
```

3. Проверка успешности операций с файлами:

```
if (!f.is_open()) {
    log_mgr.log_system("ERROR: Cannot open file");
    return;
}
```

4. Ограничение значений при приведении типов:

```
uint8_t v = static_cast<uint8_t>(max(0.0, min(255.0, map.data[y][x])));
```

Глава 6. Форматы входных и выходных файлов

6.1. Входные файлы

6.1.1. commands.txt

Файл содержит последовательность команд для выполнения.

Формат:

```
# Комментарии начинаются с #
GAUSS sign cx cy sx sy rho
GENERATE
BMP_WRITE [filename]
PLOT
PLOT2D
ANALIZ
SLOPE_CHECK
COMPONENT_SEARCH
GEOMETRY
```

Пример(мой файл):

```
# Gaussian bells for terrain generation
GAUSS 1 150 150 80 80 0.6
GAUSS 1 750 150 80 80 0.6
GAUSS 1 150 750 80 80 0.6
GAUSS 1 750 750 80 80 0.6
GAUSS 1 450 450 120 120 0.8
GAUSS -1 450 200 100 100 -0.4
GAUSS -1 200 450 100 100 -0.4
```

```
# Generate the terrain
GENERATE
```

```
# Save BMP file
BMP_WRITE my_landscape.bmp
```

```
# Create 3D visualization
PLOT
```

```
# Create 2D heatmap
PLOT2D
```

```
# Analyze slopes and gradients
ANALIZ
```

```
# Check steepness and create map
SLOPE_CHECK
```

```
# Find components using K-means
COMPONENT_SEARCH
```

```
# Build Delaunay triangulation
GEOMETRY
```

6.1.2. config.txt

Файл конфигурации в формате КЛЮЧ=ЗНАЧЕНИЕ.
Поддерживаются комментарии (#).

Пример(мой файл):

WIDTH=900

HEIGHT=900
COMMAND_FILE=commands.txt
KMEANS_K=2
SLOPE_THRESHOLD=2.0
NOISE_LEVEL=5.0
MIN_CLUSTER_SIZE=2
MAX_K=5

6.2. Выходные файлы

6.2.1. Изображения

Файл - Формат - Описание

my_landscape.bmp - BMP 8-bit - Карта высот (градации серого)
terrain_3d.png - PNG - Трехмерная визуализация
terrain_2d.png - PNG - Двумерная тепловая карта
steepness_map.bmp - BMP 8-bit - Бинарная карта крутизны
my_delaunay_voronoi.png - PNG - Триангуляция и диаграмма
Вороного

6.2.2. Данные

Файл - Содержание

terrain_data.txt - Координаты и высоты для 3D-визуализации
terrain_data_2d.txt - Координаты и высоты для 2D-визуализации
gradient_vectors.txt - Векторы градиентов (x, y, угол, модуль)
delaunay.txt - Ребра триангуляции Делоне
voronoi.txt - Ребра диаграммы Вороного
points_centers.txt - Центры компонент

6.2.3. Логи

Файл - Содержание

client_log.log - Системный лог клиента
client_user.log - Пользовательский лог клиента
server_log.log - Системный лог сервера
server_user.log - Пользовательский лог сервера

6.2.4. Скрипты

Файл - Назначение

my_plot_script.gnuplot - Скрипт для 3D-визуализации

my_plot_2d.gnuplot - Скрипт для 2D-визуализации

plot_geometry.gnuplot - Скрипт для визуализации геометрии

6.3. Формат BMP-файла

BMP (Bitmap) — простой растровый формат без сжатия. Проект создает 8-битные BMP с палитрой.

Структура заголовка файла (BITMAPFILEHEADER):

Смещение	Размер	Поле	Значение
0	2	bfType	"BM" (0x4D42)
2	4	bfSize	Размер файла
6	2	bfReserved1	0
8	2	bfReserved2	0
10	4	bfOffBits	54 + 1024 = 1078

Структура информационного заголовка (BITMAPINFOHEADER):

Смещение	Размер	Поле	Значение
14	4	biSize	40
18	4	biWidth	Ширина
22	4	biHeight	Высота
26	2	biPlanes	1
28	2	biBitCount	8
30	4	biCompression	0 (BI_RGB)
34	4	biSizeImage	Размер данных
38	4	biXPelsPerMeter	0
42	4	biYPelsPerMeter	0
46	4	biClrUsed	256
50	4	biClrImportant	0

Палитра:

256 записей по 4 байта. Для градаций серого все компоненты равны индексу.

Данные изображения:

- Строки хранятся снизу вверх
- Каждая строка выравнивается до кратности 4 байтам

Глава 7. Процесс компиляции и выполнения

7.1. Требования к системе

- Операционная система: Windows (используются Windows API для именованных каналов)
- Компилятор: Я работаю в программе CodeBlocks, компилятор C++ (поддержка C++11/17)
- Внешние зависимости: Gnuplot (должен быть установлен и доступен в PATH; у меня лежит в папке проекта)

7.2. Компиляция

Клиент автоматически:

1. Загрузит конфигурацию из config.txt
2. Прочитает команды из commands.txt
3. При необходимости запустит сервер
4. Отправит все команды серверу
5. Выведет список созданных файлов

Запуск сервера вручную

```
cmd  
terrain_generator.exe server
```

Сервер будет ожидать подключений в бесконечном цикле.

7.3. Анализ вывода компиляции

```
"C:\Users\79254\OneDrive\Рабочий стол\4 сем\gauss_with_clusters.exe"
[2026-04-19 03:13:10] SYSTEM: === Individual Project v2.7 (Client-Server + Named Pipes)
[LOG SETUP] client logs created.
[2026-04-19 03:13:10] USER: Client started
[2026-04-19 03:13:10] USER: Reading commands from: commands.txt
[2026-04-19 03:13:10] USER: Executing: GAUSS 1 150 150 80 80 0.6
[PIPE CLIENT] Server not found. Starting server...
[2026-04-19 03:13:11] USER: Executing: GAUSS 1 750 150 80 80 0.6
[2026-04-19 03:13:12] USER: Executing: GAUSS 1 150 750 80 80 0.6
[2026-04-19 03:13:12] USER: Executing: GAUSS 1 750 750 80 80 0.6
[2026-04-19 03:13:12] USER: Executing: GAUSS 1 450 450 120 120 0.8
[2026-04-19 03:13:12] USER: Executing: GAUSS -1 450 200 100 100 -0.4
[2026-04-19 03:13:12] USER: Executing: GAUSS -1 200 450 100 100 -0.4
[2026-04-19 03:13:12] USER: Executing: GENERATE
[2026-04-19 03:13:13] USER: Executing: BMP_WRITE my_landscape.bmp
[2026-04-19 03:13:13] USER: Executing: PLOT
[2026-04-19 03:13:14] USER: Executing: PLOT2D
[2026-04-19 03:13:18] USER: Executing: ANALIZ
[2026-04-19 03:13:18] USER: Executing: SLOPE_CHECK
[2026-04-19 03:13:18] USER: Executing: COMPONENT_SEARCH
[2026-04-19 03:13:20] USER: Executing: GEOMETRY
[2026-04-19 03:13:21] USER: All commands from file executed.
[2026-04-19 03:13:21] USER: All commands from commands.txt sent successfully.

=====
Program completed successfully!
Generated files:
- my_landscape.bmp
- terrain_3d.png
- terrain_2d.png
- steepness_map.bmp
- my_delaunay_voronoi.png
Check logs for details.
=====

Process returned 0 (0x0)   execution time : 10.743 s
Press any key to continue.
```

```
Выбрать C:\Users\79254\OneDrive\Рабочий стол\4 сем\gauss_with_clusters.exe
[2026-04-19 03:13:11] SYSTEM: === Individual Project v2.7 (Client-Server + Named Pipes) ===
[LOG SETUP] server logs created.
[2026-04-19 03:13:11] SYSTEM: Grid initialized 900x900
[2026-04-19 03:13:11] SYSTEM: Server initialized
[2026-04-19 03:13:11] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:11] SYSTEM: Client connected
[2026-04-19 03:13:11] SYSTEM: Received: GAUSS 1 150 150 80 80 0.6
[2026-04-19 03:13:11] SYSTEM: Added Gaussian bell at (150.000000, 150.000000)
[2026-04-19 03:13:11] SYSTEM: Client disconnected
[2026-04-19 03:13:11] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:11] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:12] SYSTEM: Client connected
[2026-04-19 03:13:12] SYSTEM: Received: GAUSS 1 750 150 80 80 0.6
[2026-04-19 03:13:12] SYSTEM: Added Gaussian bell at (750.000000, 150.000000)
[2026-04-19 03:13:12] SYSTEM: Client disconnected
[2026-04-19 03:13:12] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:12] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:12] SYSTEM: Client connected
[2026-04-19 03:13:12] SYSTEM: Received: GAUSS 1 150 750 80 80 0.6
[2026-04-19 03:13:12] SYSTEM: Added Gaussian bell at (150.000000, 750.000000)
[2026-04-19 03:13:12] SYSTEM: Client disconnected
[2026-04-19 03:13:12] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:12] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:12] SYSTEM: Client connected
[2026-04-19 03:13:12] SYSTEM: Received: GAUSS 1 750 750 80 80 0.6
[2026-04-19 03:13:12] SYSTEM: Added Gaussian bell at (750.000000, 750.000000)
[2026-04-19 03:13:12] SYSTEM: Client disconnected
[2026-04-19 03:13:12] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:12] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:12] SYSTEM: Client connected
[2026-04-19 03:13:12] SYSTEM: Received: GAUSS 1 450 450 120 120 0.8
[2026-04-19 03:13:12] SYSTEM: Added Gaussian bell at (450.000000, 450.000000)
[2026-04-19 03:13:12] SYSTEM: Client disconnected
[2026-04-19 03:13:12] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:12] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:12] SYSTEM: Client connected
[2026-04-19 03:13:12] SYSTEM: Received: GAUSS -1 450 200 100 100 -0.4
[2026-04-19 03:13:12] SYSTEM: Added Gaussian bell at (450.000000, 200.000000)
[2026-04-19 03:13:12] SYSTEM: Client disconnected
[2026-04-19 03:13:12] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:12] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:12] SYSTEM: Client connected
[2026-04-19 03:13:12] SYSTEM: Received: GAUSS -1 200 450 100 100 -0.4
[2026-04-19 03:13:12] SYSTEM: Added Gaussian bell at (200.000000, 450.000000)
[2026-04-19 03:13:12] SYSTEM: Client disconnected
[2026-04-19 03:13:12] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:12] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:12] SYSTEM: Client connected
[2026-04-19 03:13:12] SYSTEM: Received: GENERATE
[2026-04-19 03:13:13] SYSTEM: NoiseApplier: Applied 5.000000% noise (40497 pixels)
```

Выбрать C:\Users\79254\OneDrive\Рабочий стол\4 сем\gauss_with_clusters.exe

```
[2026-04-19 03:13:12] SYSTEM: Received: GENERATE
[2026-04-19 03:13:13] SYSTEM: NoiseApplier: Applied 5.000000% noise (40497 pixels)
[2026-04-19 03:13:13] SYSTEM: FieldGenerationScenario completed
[2026-04-19 03:13:13] SYSTEM: Client disconnected
[2026-04-19 03:13:13] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:13] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:13] SYSTEM: Client connected
[2026-04-19 03:13:13] SYSTEM: Received: BMP_WRITE my_landscape.bmp
[2026-04-19 03:13:13] SYSTEM: BMP saved: my_landscape.bmp
[2026-04-19 03:13:13] SYSTEM: Client disconnected
[2026-04-19 03:13:13] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:13] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:13] SYSTEM: Client connected
[2026-04-19 03:13:13] SYSTEM: Received: PLOT
[2026-04-19 03:13:14] SYSTEM: 3D Plot created: terrain_3d.png
[2026-04-19 03:13:14] SYSTEM: Client disconnected
[2026-04-19 03:13:14] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:14] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:15] SYSTEM: Client connected
[2026-04-19 03:13:15] SYSTEM: Received: PLOT2D
[2026-04-19 03:13:18] SYSTEM: 2D Heatmap created: terrain_2d.png
[2026-04-19 03:13:18] SYSTEM: Client disconnected
[2026-04-19 03:13:18] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:18] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:18] SYSTEM: Client connected
[2026-04-19 03:13:18] SYSTEM: Received: ANALYZ
[2026-04-19 03:13:18] SYSTEM: SlopeAnalysisScenario completed
[2026-04-19 03:13:18] SYSTEM: Client disconnected
[2026-04-19 03:13:18] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:18] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:18] SYSTEM: Client connected
[2026-04-19 03:13:18] SYSTEM: Received: SLOPE_CHECK
[2026-04-19 03:13:18] SYSTEM: Grid initialized 900x900
[2026-04-19 03:13:18] SYSTEM: BMP saved: steepness_map.bmp
[2026-04-19 03:13:18] SYSTEM: Steepness map created (threshold = 2.000000)
[2026-04-19 03:13:18] SYSTEM: Client disconnected
[2026-04-19 03:13:18] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:18] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:19] SYSTEM: Client connected
[2026-04-19 03:13:19] SYSTEM: Received: COMPONENT_SEARCH
[2026-04-19 03:13:20] SYSTEM: ComponentSearchScenario completed (K=2)
[2026-04-19 03:13:20] SYSTEM: Client disconnected
[2026-04-19 03:13:20] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:20] SYSTEM: Named Pipe created. Waiting for client...
[2026-04-19 03:13:20] SYSTEM: Client connected
[2026-04-19 03:13:20] SYSTEM: Received: GEOMETRY
[2026-04-19 03:13:20] SYSTEM: Dust ignored: component size = 1
[2026-04-19 03:13:20] SYSTEM: Delaunay triangles built: 8
[2026-04-19 03:13:21] SYSTEM: Geometry visualization saved: my_delaunay_voronoi.png
[2026-04-19 03:13:21] SYSTEM: GeometryScenario completed (7 centers)
[2026-04-19 03:13:21] SYSTEM: Client disconnected
[2026-04-19 03:13:21] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:21] SYSTEM: Named Pipe created. Waiting for client...
```

Рассмотрим подробный лог серверной части программы, который демонстрирует полный цикл обработки команд.

1. Инициализация сервера

[2026-04-19 03:13:11] SYSTEM: === Individual Project v2.7 (Client-Server + Named Pipes) ===

[LOG SETUP] server logs created.

[2026-04-19 03:13:11] SYSTEM: Grid initialized 900x900

[2026-04-19 03:13:11] SYSTEM: Server initialized

[2026-04-19 03:13:11] SYSTEM: Named Pipe created. Waiting for client...

Сервер успешно стартовал, выделил память под карту размером 900×900 пикселей и перешел в режим ожидания подключения клиента через именованный канал.

2. Обработка команд GAUSS

[2026-04-19 03:13:11] SYSTEM: Client connected
[2026-04-19 03:13:11] SYSTEM: Received: GAUSS 1 150 150 80 80 0.6
[2026-04-19 03:13:11] SYSTEM: Added Gaussian bell at (150.000000, 150.000000)
[2026-04-19 03:13:11] SYSTEM: Client disconnected
[2026-04-19 03:13:11] SYSTEM: Pipe closed. Ready for new client.

Сервер последовательно получает команды на добавление гауссовых колоколов. Для каждой команды создается отдельное соединение: клиент подключается, отправляет команду, получает подтверждение и отключается. Сервер при этом сохраняет состояние между подключениями, накапливая список колоколов в векторе bells. В логе видно добавление семи колоколов с различными параметрами, включая корреляцию ($\rho = 0.6$, $\rho = 0.8$, $\rho = -0.4$).

3. Генерация ландшафта

[2026-04-19 03:13:12] SYSTEM: Received: GENERATE
[2026-04-19 03:13:13] SYSTEM: NoiseApplier: Applied 5.000000% noise (40497 pixels)
[2026-04-19 03:13:13] SYSTEM: FieldGenerationScenario completed

При получении команды GENERATE запускается сценарий генерации ландшафта. Выполняется суперпозиция всех добавленных гауссовых колоколов, нормализация высот в диапазон $[0, 255]$, добавление шума (затронуто 40497 пикселей из 810000, что соответствует заданным 5%) и двойное сглаживание гауссовым фильтром.

4. Сохранение BMP и визуализация

[2026-04-19 03:13:13] SYSTEM: Received: BMP_WRITE my_landscape.bmp
[2026-04-19 03:13:13] SYSTEM: BMP saved: my_landscape.bmp

[2026-04-19 03:13:13] SYSTEM: Received: PLOT
[2026-04-19 03:13:14] SYSTEM: 3D Plot created: terrain_3d.png

[2026-04-19 03:13:15] SYSTEM: Received: PLOT2D
[2026-04-19 03:13:18] SYSTEM: 2D Heatmap created: terrain_2d.png

Сервер последовательно создает все визуализации: 8-битное BMP-изображение карты высот, трехмерную визуализацию и двумерную

тепловую карту. Для построения графиков используются скрипты gnuplot, которые генерируются автоматически.

5. Анализ градиентов и крутизны склонов

[2026-04-19 03:13:18] SYSTEM: Received: ANALIZ

[2026-04-19 03:13:18] SYSTEM: SlopeAnalysisScenario completed

[2026-04-19 03:13:18] SYSTEM: Received: SLOPE_CHECK

[2026-04-19 03:13:18] SYSTEM: Grid initialized 900x900

[2026-04-19 03:13:18] SYSTEM: BMP saved: steepness_map.bmp

[2026-04-19 03:13:18] SYSTEM: Steepness map created (threshold = 2.000000)

Команда ANALIZ вычисляет градиенты во всех точках ландшафта и сохраняет векторы в файл gradient_vectors.txt. Команда SLOPE_CHECK создает бинарную карту крутизны: белым отмечаются пологие участки (крутизна < 2.0), черным — крутые склоны.

6. Кластеризация и поиск компонент

[2026-04-19 03:13:19] SYSTEM: Received: COMPONENT_SEARCH

[2026-04-19 03:13:20] SYSTEM: ComponentSearchScenario completed (K=2)

Выполняется кластеризация высоких участков ландшафта (высота ≥ 128) методом К-средних с параметром K=2. Алгоритм итеративно разделяет множество точек на два кластера, минимизируя сумму квадратов расстояний до центров.

7. Геометрический анализ

[2026-04-19 03:13:20] SYSTEM: Received: GEOMETRY

[2026-04-19 03:13:20] SYSTEM: Dust ignored: component size = 1

[2026-04-19 03:13:20] SYSTEM: Delaunay triangles built: 8

[2026-04-19 03:13:21] SYSTEM: Geometry visualization saved: my_delaunay_voronoi.png

[2026-04-19 03:13:21] SYSTEM: GeometryScenario completed (7 centers)

При выполнении команды GEOMETRY происходит следующее:

- Поиск связных компонент с порогом высоты > 200

- Фильтрация мелких компонент: одиночные пиксели (размером 1) игнорируются как шум
- Вычисление центров найденных компонент (в данном случае найдено 7 центров)
- Построение триангуляции Делоне по центрам (построено 8 треугольников)
- Построение диаграммы Вороного на основе смежных треугольников
- Визуализация геометрических структур в файл my_delaunay_voronoi.png

8. Завершение сессии

```
[2026-04-19 03:13:21] SYSTEM: Client disconnected
[2026-04-19 03:13:21] SYSTEM: Pipe closed. Ready for new client.
[2026-04-19 03:13:21] SYSTEM: Named Pipe created. Waiting for client...
```

После выполнения всех команд клиент отключается, однако сервер продолжает работу и снова создает именованный канал, ожидая новых подключений. Это позволяет выполнить несколько сеансов генерации без перезапуска сервера.

Глава 8. Практические примеры и анализ результатов

8.1. Базовый ландшафт с семью гауссовыми колоколами

Используется файл commands.txt со следующим набором команд:

```
GAUSS 1 150 150 80 80 0.6
GAUSS 1 750 150 80 80 0.6
GAUSS 1 150 750 80 80 0.6
GAUSS 1 750 750 80 80 0.6
GAUSS 1 450 450 120 120 0.8
GAUSS -1 450 200 100 100 -0.4
GAUSS -1 200 450 100 100 -0.4
```

```
GENERATE
BMP_WRITE my_landscape.bmp
```

PLOT
PLOT2D
ANALIZ
SLOPE_CHECK
COMPONENT_SEARCH
GEOMETRY

8.1.1. Анализ параметров гауссовых колоколов

Рассмотрим подробно каждый из семи колоколов, формирующих ландшафт:

№	Колокол	Центр (c_x, c_y)	σ_x, σ_y	ρ	Тип	Описание
1	GAUSS 1	(150, 150)	80, 80	0.6	Холм	Угловой холм в левом нижнем углу с положительной корреляцией
2	GAUSS 1	(750, 150)	80, 80	0.6	Холм	Угловой холм в правом нижнем углу с положительной корреляцией
3	GAUSS 1	(150, 750)	80, 80	0.6	Холм	Угловой холм в левом верхнем углу с положительной корреляцией
4	GAUSS 1	(750, 750)	80, 80	0.6	Холм	Угловой холм в правом верхнем углу с положительной корреляцией
5	GAUSS 1	(450, 450)	120, 120	0.8	Холм	Центральный пик с сильной положительной корреляцией
6	GAUSS - 1	(450, 200)	100, 100	-0.4	Впадина	Впадина в верхней части карты с отрицательной корреляцией

7	GAUSS - 1	(200, 450)	100, 100	-0.4	Впадина	Впадина в левой части карты с отрицательной корреляцией
---	--------------	------------	----------	------	---------	---

8.1.2. Влияние корреляции на форму рельефа

Коэффициент корреляции ρ существенно влияет на ориентацию эллипса уровня гауссова колокола:

- Положительная корреляция ($\rho > 0$) вытягивает колокол вдоль главной диагонали (направление вектора $(1, 1)$, угол $+45^\circ$). Контуры становятся эллипсами, вытянутыми по линии $y = x$.
- Отрицательная корреляция ($\rho < 0$) вытягивает колокол вдоль антидиагонали (направление вектора $(1, -1)$, угол -45° или 135°). Контуры вытянуты по линии $y = -x$.

Таким образом:

- Все положительные колокола ($\rho = 0.6$ и $\rho = 0.8$) вытянуты примерно по диагонали слева-снизу $\rightarrow$ справа-сверху ($y = x$).
- Оба отрицательных колокола ($\rho = -0.4$) вытянуты по диагонали слева-сверху $\rightarrow$ справа-снизу ($y = -x$), что придаёт им характерную «седловидную» или косую форму.

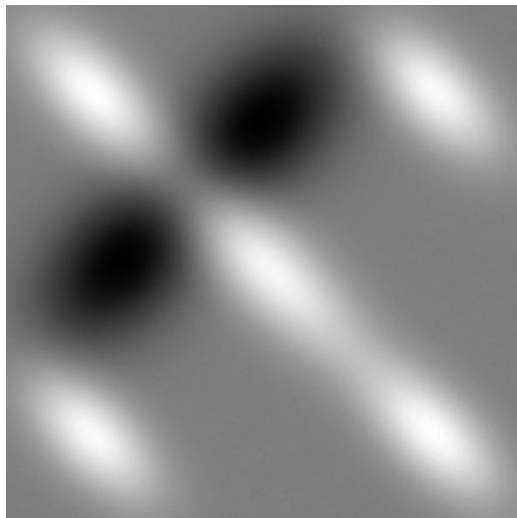
Чем больше $|\rho|$, тем сильнее вытянутость (при $\rho = 0.8$ вытянутость заметно сильнее, чем при $\rho = 0.6$).

- При $\rho > 0$ (0.6 и 0.8) колокол вытягивается вдоль диагонали $y = x$ (направление $+45^\circ$).
- При $\rho < 0$ (-0.4) колокол вытягивается вдоль диагонали $y = -x$ (направление -45°).

В результате угловые холмы и центральный пик вытянуты по одной диагонали, а впадины — по перпендикулярной, что создаёт интересное пересечение структур и зоны повышенных градиентов в местах их взаимодействия.

8.2. Результаты визуализации

8.2.1. Карта высот (my_landscape.bmp)

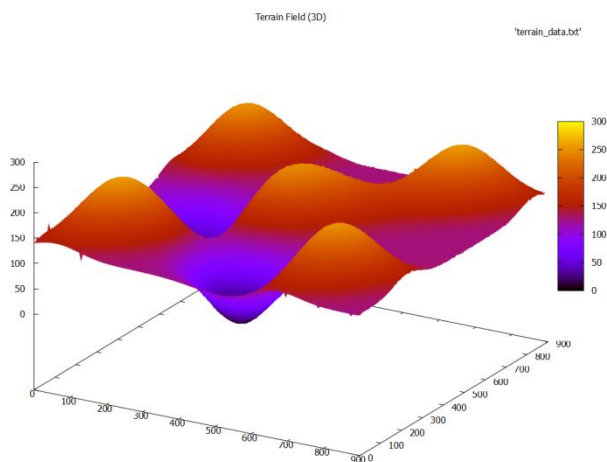


Выходное BMP-изображение представляет собой 8-битную карту высот в градациях серого:

- Светлые области (ближе к 255) — возвышенности и пики
- Темные области (ближе к 0) — низины и впадины
- Средние тона — переходные зоны и равнинные участки

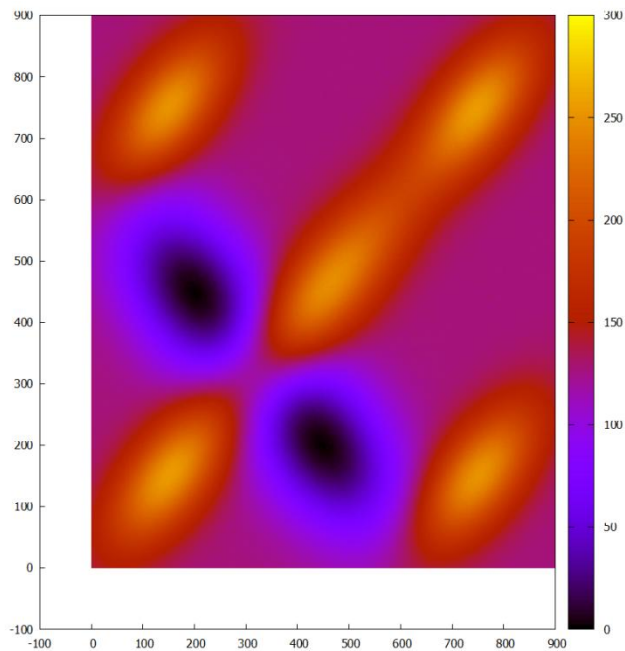
На изображении отчетливо видны светлые пятна и две затемненные области в верхней и левой частях карты.

8.2.2. Трехмерная визуализация (terrain 3d.png)



3D-визуализация, построенная с помощью `gnuplot`, позволяет оценить рельеф в перспективе.

8.2.3. Тепловая карта (terrain_2d.png)



Двумерная тепловая карта использует цветовую шкалу для отображения высот:

- Красные и желтые тона — высокие участки
- Розовые тона — средние высоты
- Чёрные и синие тона — низменности

Такое представление удобно для быстрой оценки распределения высот и выявления локальных экстремумов.

8.3. Анализ поля, градиентов и крутизны

8.3.1. Векторы градиентов

Файл `gradient_vectors.txt` содержит для каждой точки (с шагом 20 пикселей) следующие данные:

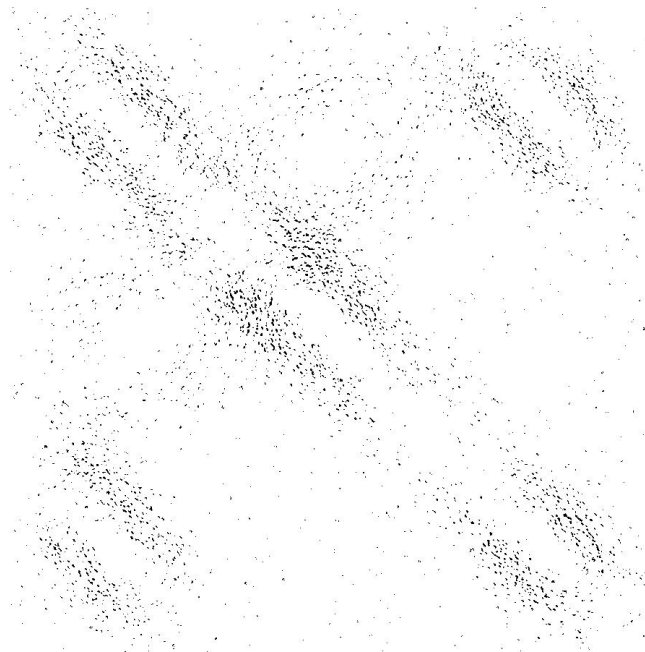
- Координаты (x, y)
- Угол наклона в радианах
- Модуль градиента (крутизна)

Анализ этих данных показывает, что:

- На вершинах холмов и в центре впадин модуль градиента близок к нулю
- На склонах центрального гребня крутизна достигает максимальных значений

- Углы наклона на склонах угловых холмов направлены преимущественно к центру карты

8.3.2. Карта крутизны склонов



Файл `steepness_map.bmp` представляет собой бинарное изображение:

- Белые пиксели (255): пологие участки с крутизной < 2.0
- Черные пиксели (0): крутые склоны с крутизной ≥ 2.0

При заданном пороге 2.0:

- Вершины холмов и днища впадин отображаются белым
- Склоны центрального гребня и переходные зоны — черным
- Наиболее крутые участки расположены на границах между холмами и впадинами

8.3.3. State Machine Diagram

Основные состояния (State):

- Initial / RawField
- GeneratedField (составное состояние — самое важное)
 - Внутри: NoisyField, SmoothedField, VisualizedField, BMPExists
- Slope Twin (Порог наклона)
- Binary Twin (Высотный порог)

- Field with Clusters
- ClustersFound
- GeometryReady + DelaunayVisual

Выполнена в PlantUML:

```
@startuml
skinparam shadowing false
skinparam state {
    BackgroundColor #E3F2FD
    BorderColor #1976D2
}

[*] --> Initial

state Initial {
    [*] --> RawField : Добавление гауссовых колоколов
    RawField --> GeneratedField : Генерация + нормализация
}

state GeneratedField {
    state "Noisy Field" as Noisy
    state "Smoothed Field" as Smoothed
    state "Visualized Field" as Visualized
    state "BMP Saved" as BMP

    [*] --> Noisy : Добавление шума
    Noisy --> Smoothed : Сглаживание (x2)
    Smoothed --> Visualized : Визуализация 3D/2D (gnuplot)
    Smoothed --> BMP : Сохранение в BMP
    Visualized --> GeneratedField : Повторная визуализация
    BMP --> GeneratedField : Повторное сохранение
}

GeneratedField --> SlopeTwin : Расчёт градиентов и наклонов
GeneratedField --> BinaryTwin : Бинаризация по высоте (> 200)

state "Slope Twin\n(Порог наклона)" as SlopeTwin
state "Binary Twin\n(Высотный порог)" as BinaryTwin

SlopeTwin --> SlopeVisual : Визуализация градиентов
SlopeTwin --> BinaryTwin : Применение порога по наклону

BinaryTwin --> ClusterField : Кластеризация + Поиск связанных компонент

state "Field with Clusters" as ClusterField
state "Clusters Found" as ClustersFound

ClusterField --> ClustersFound : Сборка списка кластеров

ClustersFound --> GeometryReady : Триангуляция Делоне + диаграмма Вороного

state GeometryReady {
    [*] --> DelaunayVisual : Построение и визуализация
    DelaunayVisual --> [*] : Завершение геометрического анализа
}
```

}

note right of GeneratedField

Основное рабочее состояние поля

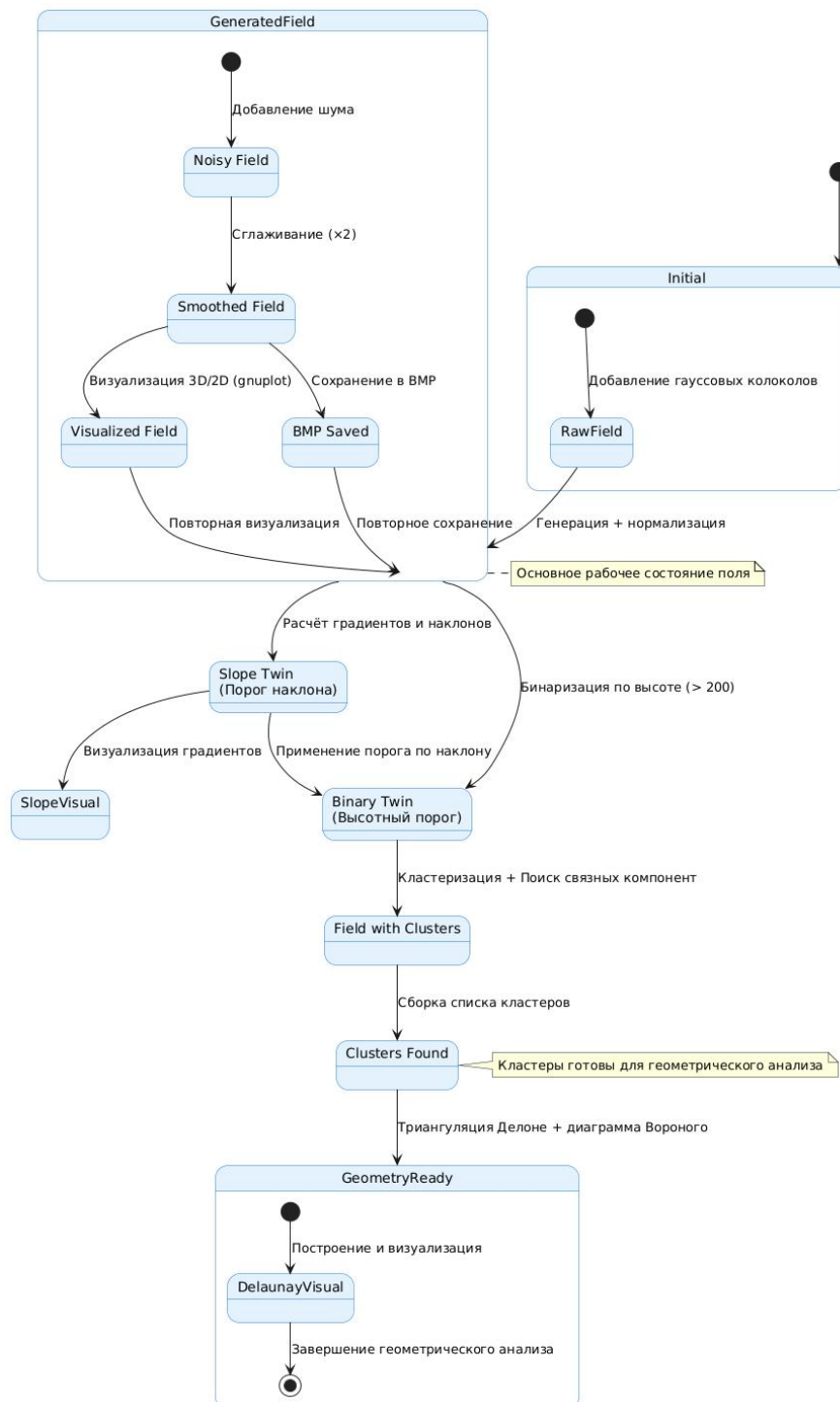
end note

note right of ClustersFound

Кластеры готовы для геометрического анализа

end note

@enduml



8.4. Кластеризация методом K-средних

Алгоритм K-средних применяется к точкам с высотой ≥ 128 (верхняя половина диапазона яркости).

Параметры кластеризации:

- Количество кластеров $K = 2$
- Минимальный размер кластера: 2 пикселя

Результаты:

- Множество высоких точек разделяется на два крупных кластера
- Граница между кластерами проходит примерно по диагонали карты
- Первый кластер объединяет преимущественно левую и верхнюю части
- Второй кластер охватывает правую и нижнюю части

Кластеризация позволяет автоматически выделить макрорегионы с повышенной высотой.

8.5. Поиск связных компонент и геометрический анализ

8.5.1. Поиск связных компонент

На предварительном этапе геометрического анализа выполняется поиск связных компонент — областей с высотой > 200 , соединенных по 8-связности.

Результаты поиска:

- Найдено несколько значимых компонент, соответствующих вершинам холмов
- Мелкие компоненты размером 1 пиксель отфильтровываются как шум (в логе: Dust ignored: component size = 1)
- После фильтрации остается 7 центров компонент

8.5.2. Триангуляция Делоне

По 7 найденным центрам строится триангуляция Делоне.

Характеристики триангуляции:

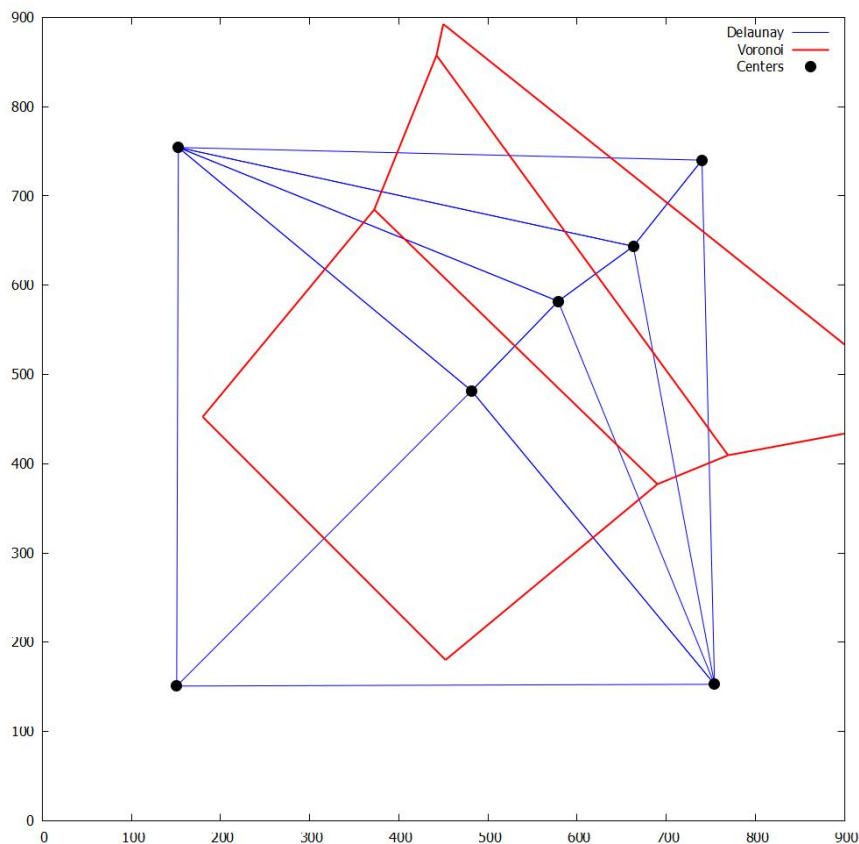
- Количество точек: 7
- Количество треугольников: 8

Треугольники Делоне обладают свойством максимальной минимальности углов, что обеспечивает «качественное» разбиение без чрезмерно вытянутых форм.

8.5.3. Диаграмма Вороного

На основе триангуляции Делоне строится диаграмма Вороного. Ребра диаграммы соединяют центры описанных окружностей смежных треугольников (имеющих две общие вершины).

8.5.4. Визуализация геометрии



Файл `my_delaunay_voronoi.png` содержит:

- Синие линии — ребра триангуляции Делоне
- Красные линии — ребра диаграммы Вороного
- Черные точки — центры связных компонент

Визуализация наглядно демонстрирует двойственность двух геометрических структур и позволяет оценить пространственное расположение ключевых точек ландшафта.

8.6. Статистика выполнения

Из лога выполнения можно извлечь следующую статистику:

Показатель - Значение

Размер карты - $900 \times 900 = 810\,000$ пикселей

Количество гауссовых колоколов - 7

Затронуто шумом (5%) $\approx 40\,500$

Количество центров компонент - 7

Количество треугольников Делоне - 8

Порог крутизны - 2.0

Количество кластеров K-means - 2

Общее время выполнения - ~ 10.7 секунд

8.7. Интерпретация результатов

Анализ полученных результатов позволяет сделать следующие выводы:

1. Коррелированные гауссианы создают более сложный и реалистичный рельеф по сравнению с изотропными ($\rho = 0$). Вытянутые формы имитируют горные хребты и долины.
2. Комбинация положительных и отрицательных колоколов позволяет моделировать не только возвышенности, но и впадины, что существенно расширяет выразительные возможности генератора.

2. Геометрический анализ выявляет 7 ключевых точек ландшафта, которые могут интерпретироваться как «вершины» и служить основой для дальнейшего моделирования.
4. Фильтрация мелких компонент эффективно устраняет шумовые артефакты, оставляя только значимые элементы рельефа.
5. Визуализация геометрических структур обеспечивает наглядное представление о пространственных отношениях между ключевыми точками ландшафта.

Глава 9. Заключение

9.1. Обзор достигнутых результатов

В рамках данного проекта была разработана система для генерации, визуализации и анализа искусственного ландшафта. Ключевые достижения:

1. Математическая модель: Использование суперпозиции двумерных гауссовых функций с корреляцией позволяет создавать разнообразные формы рельефа — от одиночных холмов до сложных горных систем.
2. Клиент-серверная архитектура: Применение именованных каналов Windows.
3. Комплексный анализ: Реализованы алгоритмы вычисления градиентов, кластеризации К-средних, поиска связных компонент, построения триангуляции Делоне и диаграммы Вороного.
4. Множественные форматы вывода: Система генерирует BMP-изображения, PNG-визуализации через `gnuplot`, текстовые файлы с данными для дальнейшего анализа.
5. Подробное логирование: Двухуровневая система логирования обеспечивает прозрачность работы и упрощает отладку.

Приложение А: Структура файлов проекта

— gauss_with_clusters.cpp	# Основной исходный файл
— config.txt	# Файл конфигурации
— commands.txt	# Файл команд
— client_log.log	# Системный лог клиента
— client_user.log	# Пользовательский лог клиента
— server_log.log	# Системный лог сервера
— server_user.log	# Пользовательский лог сервера
— my_landscape.bmp	# Карта высот
— terrain_3d.png	# 3D-визуализация
— terrain_2d.png	# 2D-тепловая карта
— steepness_map.bmp	# Карта крутизны
— my_delaunay_voronoi.png	# Геометрическая визуализация
— terrain_data.txt	# Данные для 3D
— terrain_data_2d.txt	# Данные для 2D
— gradient_vectors.txt	# Векторы градиентов
— delaunay.txt	# Ребра Делоне
— voronoi.txt	# Ребра Вороного
— points_centers.txt	# Центры компонент
— my_plot_script.gnuplot	# Скрипт 3D
— my_plot_2d.gnuplot	# Скрипт 2D
— plot_geometry.gnuplot	# Скрипт геометрии